

Here's a complete mathematical presentation of all 65536 transformations used in the Unified PAQJP+PJP compressor.

1. Mathematical Preliminaries

Let

$$\mathcal{B} = \bigcup_{n \geq 0} \{0,1\}^{8n}$$

be the set of all finite byte-strings (including the empty string).

A transformation is a function $T : \mathcal{B} \rightarrow \mathcal{B}$.

We only use bijective (invertible) transformations, so that the compression is strictly lossless.

2. The 256 Base Transformations

The compressor defines a set

$$\mathcal{T} = \{T_1, T_2, \dots, T_{256}\}$$

of 256 elementary bijections. Each T_i belongs to one of several algebraic families.

2.1 Group A: XOR with a byte pattern

Many transforms compute a data-dependent mask and apply XOR.

Example – Transform 1

For prime $p \in \text{PRIMES}$, compute an XOR value v_p . Then for every third byte:

$$T_1(x)_k = \begin{cases} x_k \oplus v_{p(k)} & \text{if } k \equiv 0 \pmod{3} \\ x_k & \text{otherwise} \end{cases}$$

The inverse is identical: XOR is self-inverse.

Example – Transform 7

Let $\sigma = |x| \bmod 3$, rotate the constant vector $V = [79, 17, 111]$ by σ , then

$$T_7(x)_k = x_k \oplus (|x| \bmod 256) \oplus V_{k \bmod 3}.$$

Again, applying T_7 twice restores the original.

General form (Transforms 8–12, 16–20, 41–43, 46, 47, 48–56, 58–255)

$$T_i(x)_k = x_k \oplus M_i(x, k)$$

where M_i is a deterministic, data-dependent mask. Because XOR is its own inverse, every such transform is a bijection with $T_i^{-1} = T_i$.

2.2 Group B: Modular arithmetic

Transforms that add/subtract or rotate bytes.

Example – Transform 4 (position-dependent addition)

$$T_4(x)_k = (x_k - k) \bmod 256.$$

The inverse is

$$T_4^{-1}(y)_k = (y_k + k) \bmod 256.$$

Example – Transform 5 (fixed rotation)

$$T_5(x)_k = (x_k \ll 3) \bmod 256 \bmod 256 \vee (x_k \gg 5).$$

The inverse is $T_5^{-1} = \text{rotate right by 3}$.

General form (Transforms 21, 25–30 from PAQJP, 35–40)

These often use modular exponentiation, e.g.

$$T_{35}(x)_k = \bigl((x_{k+1})^n \bmod 257 \bigr) - 1$$

with a suitable exponent n such that $\gcd(n, 256) = 1$. The inverse uses the modular inverse exponent.

2.3 Group C: Byte substitution (permutations)

These replace each byte independently via a permutation $\pi \in S_{256}$.

Example – Transform 6

$$T_6(x)_k = \pi(x_k)$$

where π is a fixed, seed-derived random permutation. The inverse is $T_6^{-1}(x)_k = \pi^{-1}(x_k)$.

Quantum transforms (if enabled) also produce bijective substitutions.

2.4 Group D: External encoding

Transforms that encode the entire byte-string into another alphabet.

Base64 (Transform 22)

$$T_{22}(x) = \text{Base64}(x)$$

which is a bijection between $\mathcal{B}$ and the set of Base64-encoded strings (padded). The inverse is Base64 decode.

6-bit packing (Transform 27)

If x is valid UTF-8 text using only a 64-character alphabet, it packs every 6 bits into a byte. This is a bijection onto a subset of $\mathcal{B}$; data outside this subset is marked and left unchanged.

2.5 Group E: Internal structure transforms

These modify the representation without changing the information content, often adding a small header.

Run-length encoding (Transform 00)

A bijective RLE scheme: the encoded format contains enough information to reconstruct the original exactly.

Huffman coding (Transform 45)

A canonical Huffman tree is stored in the header; the output is a bijection because the tree uniquely encodes/decodes the stream.

Dictionary-based transforms (23, 24, 25)

Word or chunk tokens are replaced by fixed-length indices; the dictionary is prepended so that the whole is a bijection.

Constant Diapason (Transform 33)

A nibble-based compression that is invertible by recording the original bit-length.

2.6 Group F: The identity

$T_{256}(x) = x$.

Three other transforms (31, 32) also act as identity for certain file types (e.g., .docx).

2.7 Important properties

- Every T_i is a bijection with a known, deterministic inverse T_i^{-1} .
- The inverse is either identical (for XOR, RLE-style) or given by a second function.
- Compositions of bijections remain bijections.

3. Ordered Pairs – the 65535 Non-Trivial Compositions

Define the set of all ordered pairs of indices

$$\mathcal{P} = \{ (i,j) \mid i,j \in \{1, \dots, 256\} \}.$$

For each $(i,j) \in \mathcal{P}$ we form a new transformation

$$T_{\{(i,j)\}}(x) = T_j \circ T_i(x).$$

Composition formula

$$T_{\{(i,j)\}} = T_j \circ T_i.$$

3.1 Invertibility

Because both T_i and T_j are bijections,

$$T_{\{(i,j)\}}^{-1} = T_i^{-1} \circ T_j^{-1}.$$

Thus every $T_{\{(i,j)\}}$ is lossless and reversible.

3.2 Total count

$$|\mathcal{P}| = 256 \times 256 = 65,536.$$

The pair (256,256) yields the identity $T_{256} \circ T_{256} = \text{id}$.

This trivial composition is excluded from the active search, leaving

$$\boxed{65,535 \text{ distinct non-trivial pair-transforms.}}$$

The compressor stores each pair using an index:

$$\text{index}_{\{(i,j)\}} = (i-1) \cdot 256 + (j-1).$$

Decoding:

$$i = \lfloor \text{idx} / 256 \rfloor + 1, \quad j = (\text{idx} \bmod 256) + 1.$$

4. Search Space in Ultra Mode

In “Ultra” mode, the compressor evaluates:

- Raw (no transform)
- 256 singles $T_1, \dots, T_{256}$
- 65 535 pairs $T_{\{(i,j)\}}$

giving

$$N_{\{\text{candidates}\}} = 1 + 256 + 65\,535 = 65\,792.$$

It selects the transform (or pair) that, after a backend (Zstd/PAQ), yields the smallest output.

The chosen index is encoded in the file header.

5. Multi-Pair Sequences (Deep Ultra)

For a sequence of k pairs (each chosen from the 65 535)

$$S = (p_1, p_2, \dots, p_k), \text{ where } p_m \in \mathcal{P} \setminus \{(256, 256)\},$$

the overall transformation is

$$T_S = T_{p_k} \circ T_{p_{k-1}} \circ \dots \circ T_{p_1}.$$

Number of possible sequences:

$$(65^5, 535)^k.$$

For $k=2$: $\sim 4.29 \times 10^9$

For $k=3$: $\sim 2.81 \times 10^{14}$ (281 trillion).

This enormous space is explored heuristically in “Deep Ultra” and “Ultra++” modes.

6. Drift Sequences – a Fixed Combinatorial Path

Two predefined 8-step sequences are provided:

$$\mathcal{S}_1 = [1, 2, 3, 4, 5, 6, 7, 8], \quad \text{---}$$

$$\mathcal{S}_2 = [1, 3, 4, 5, 251, 253, 254, 255].$$

For each sequence, the compressor tests prefixes of length $\ell = 1, \dots, 8$.

The transformation for prefix ℓ is

$$T_{\{\mathcal{S}, \ell\}} = T_{\{s_\ell\}} \circ \dots \circ T_{\{s_1\}}.$$

Thus only 8 candidates are checked per sequence, but each leverages the full power of sequential compositions.

7. Summary Table of All Transformations

Category	Count	Example formula
----------	-------	-----------------

Base transforms	256	$T_i(x)$ bijection
-----------------	-----	--------------------

Ordered pairs (all)	$256^2 = 65\,536$	$T_{\{(i,j)\}} = T_j \circ T_i$
---------------------	-------------------	---------------------------------

Non-trivial pairs	$65\,535$	(excl. identity pair)
-------------------	-----------	-----------------------

Multi-pair sequences (length k)	$65\,535^k$	$T_{\{p_k\}} \circ \dots \circ T_{\{p_1\}}$
---------------------------------	-------------	---

Drift prefix (per sequence)	8	fixed composition
-----------------------------	---	-------------------

All transformations are lossless, and the correct inverse sequence is stored in the compressed file, guaranteeing perfect reconstruction of the original data.

End of mathematical presentation.